




New Mansoura University
Faculty of Computer Science and Engineering




# Advanced Machine Learning

## Regularization

**Islam Reda**

Associate Professor

islam.reda@nmu.edu.eg

# Outline

- Polynomial Regression
- Regularized Linear Models
  - Ridge Regression
  - Lasso Regression
  - Elastic Net

# Polynomial Regression

# Polynomial Regression

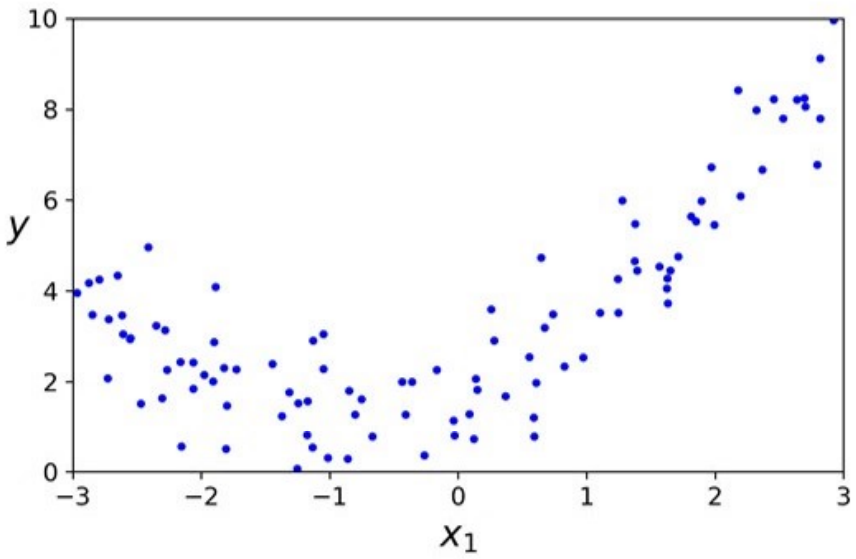
- What if your data is actually more complex than a simple straight line?
- You can actually **use a linear model to fit nonlinear data**.
- To do this:
  - Add powers of each feature as new features,
  - Then train a linear model on this extended set of features.
- This technique is called **Polynomial Regression**.

# Polynomial Regression

- First, let's generate some nonlinear data, based on a simple quadratic equation:

```
m = 100
X = 6 * np.random.rand(m, 1) - 3
y = 0.5 * X**2 + X + 2 + np.random.randn(m, 1)
```

- A straight line will never fit this data properly.
- Use PolynomialFeatures class to transform our training data, adding the square of each feature in the training set as new features




Generated nonlinear and noisy dataset

# Polynomial Regression

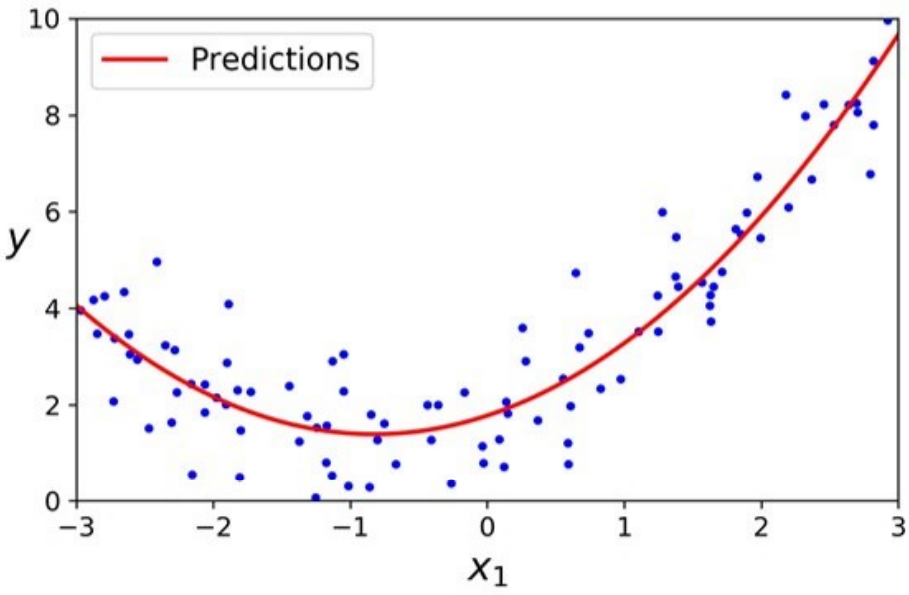
```
>>> from sklearn.preprocessing import PolynomialFeatures
>>> poly_features = PolynomialFeatures(degree=2, include_bias=False)
>>> X_poly = poly_features.fit_transform(X)
>>> X[0]
array([-0.75275929])
>>> X_poly[0]
array([-0.75275929, 0.56664654])
```

- X_poly now contains the original feature of X plus the square of this feature. Fit a LinearRegression model to this extended training data.

```
>>> lin_reg = LinearRegression()
>>> lin_reg.fit(X_poly, y)
>>> lin_reg.intercept_, lin_reg.coef_
(array([1.78134581]), array([[0.93366893, 0.56456263]]))
```

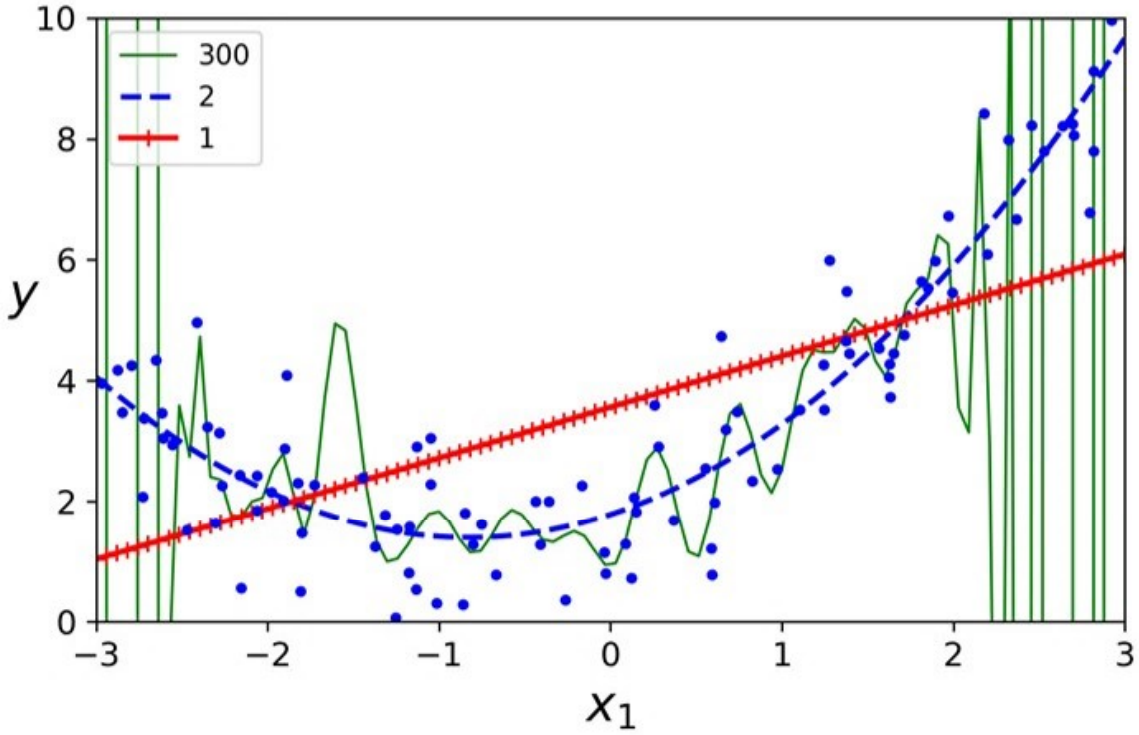
# Polynomial Regression

- Model estimates
  $\hat{y} = 0.56x_1^2 + 0.93x_1 + 1.78$
- PolynomialFeatures also adds all combinations of features up to the given degree.
  - If there were two features $a$ and $b$, PolynomialFeatures with degree=3 would not only add the features $a^2, a^3, b^2$, and $b^3$, but also the combinations $ab$, $a^2b$, and $ab^2$.




Polynomial Regression model predictions

# High-degree Polynomial Regression




High-degree Polynomial Regression

# High-degree Polynomial Regression

- How can you tell that your model is overfitting or underfitting the data?
- Use **cross-validation** to get an estimate of a model's **generalization performance**.
  - If a model **performs well on** the **training data** but **generalizes poorly** according to the cross-validation metrics, then your **model is overfitting**.
  - If a model **performs poorly on both**, then it is **underfitting**.

# High-degree Polynomial Regression

- If your model is **underfitting** the training data, **adding more training examples will not help**.
- You need to **use a more complex model** or **come up with better features**.

- To improve an **overfitting model**, **feed it more training data** until the validation error reaches the training error.

# Regularized Linear Models

# Regularized Linear Models

- A good way **to reduce overfitting** is to **regularize the model** (i.e., to **constrain** it):
  - To **regularize a polynomial model**, **reduce the number of polynomial degrees**.
  - **For a linear model**, regularization is typically achieved by **constraining the weights** of the model.
    - Ridge Regression
    - Lasso Regression
    - Elastic Net

# Ridge (L2) Regression

- A **regularization term** ($\alpha \sum_{j=1}^{n} \theta_j^2$) is added to the cost function.
- This forces the learning algorithm to not only **fit the data** but also **keep** the model **weights** as **small** as possible.
- The **regularization term** should only be **added** to the cost function **during training**.
- Once the model is trained, you want to **evaluate the model**'s performance using the **unregularized performance measure**.

# Ridge (L2) Regression

- Ridge regression cost function:

$$J(\boldsymbol{\theta}) = \mathrm{MSE}(\boldsymbol{\theta}) + \alpha \frac{1}{2} \sum_{j=1}^{n} \theta_j^2$$

- The hyperparameter $\alpha$ controls how much you want to regularize the model
  - If $\boldsymbol{\alpha = 0}$, then ridge regression is just linear regression.
  - If $\boldsymbol{\alpha}$ is very **large**, then **all weights** end up **very close to zero** and the result is a flat line going through the data's mean.
- It is important to **scale the data before performing ridge regression**, as it is sensitive to the scale of the input features

# Ridge (L2) Regression

- To perform ridge regression using stochastic gradient descent:

```
>>> sgd_reg = SGDRegressor(penalty="l2")
>>> sgd_reg.fit(X, y.ravel())
>>> sgd_reg.predict([[1.5]])
array([1.47012588])
```

- The penalty hyperparameter sets the type of regularization term to use. Specifying "l2" indicates that you want SGD to use ridge regression.

# Lasso (L1) Regression

- Least Absolute Shrinkage and Selection Operator (Lasso) Regression
- It adds a **regularization term** (the $\ell_1$ **norm** of the weight vector) to the cost function.
- Lasso regression cost function:

$$J(\boldsymbol{\theta}) = \mathrm{MSE}(\boldsymbol{\theta}) + \alpha \sum_{j=1}^{n} |\theta_j|$$

- Lasso regression ends to completely **eliminate** the **weights** of the **least important features** (i.e., set them to zero).
- It **automatically performs feature selection** and outputs a sparse model (i.e., with few nonzero feature weights).

# Lasso (L1) Regression

- A small Scikit-Learn example using the **Lasso** class. Note that you could instead use an **SGDRegressor(penalty="l1")**.

```
>>> from sklearn.linear_model import Lasso
>>> lasso_reg = Lasso(alpha=0.1)
>>> lasso_reg.fit(X, y)
>>> lasso_reg.predict([[1.5]])
array([1.53788174])
```

# Elastic Net

- The regularization term is a simple mix of both Ridge and Lasso's regularization terms, and you can control the mix ratio $r$.
- Elastic net cost function:

$$J(\boldsymbol{\theta}) = \mathrm{MSE}(\boldsymbol{\theta}) + r\alpha \sum_{j=1}^{n} \left|\theta_j\right| + \frac{1-r}{2} \alpha \sum_{j=1}^{n} \theta_j^2$$

- When $r = 0$, Elastic net ≡ ridge regression
- When $r = 1$, Elastic net ≡ lasso regression

# Elastic Net

- Scikit-Learn's ElasticNet (l1_ratio corresponds to the mix ratio $r$).

```
>>> from sklearn.linear_model import ElasticNet
>>> elastic_net = ElasticNet(alpha=0.1, l1_ratio=0.5)
>>> elastic_net.fit(X, y)
>>> elastic_net.predict([[1.5]])
array([1.54333232])
```

# Regularized Linear Models

- When should you use plain linear regression (i.e., without any regularization), ridge, lasso, or elastic net?
  - Almost always **preferable to have** at least a little bit of **regularization** (**avoid plain linear regression**).
  - **Ridge** is a **good default**.
  - If you suspect that **only a few features are** actually **useful**, you should prefer **lasso or elastic net**.
  - **Elastic net** is **preferred over lasso**.

# The Bias/Variance Tradeoff

- **Bias**: Error due to **wrong assumptions**, such as assuming that the data is linear when it is actually quadratic. A **high-bias model underfits** the training data.
- **Variance**: Error due to the model's **excessive sensitivity to small variations in the training data**. A model with many degrees of freedom is likely to have high variance, and thus to **overfit** the training data.
- **Increasing** a model's **complexity** will typically **increase** its **variance** and **reduce** its **bias** and vice versa.
- This is why it is called a **tradeoff**.

# Thank You